Review of "Intermittent working-memory coding through NMDA-sustained persistent activity"

Reviewed: the Google Doc manuscript (version of 2026-11-01, all of Tab 1 read, incl. Methods and supplementary captions) and this repository at `cd1a59e` (`main`). Everything in `review/` is new; nothing in `nmdaint/`, `scripts/`, `results/` or the notebooks was changed. Each claim below names the script that tests it and the file in `review/out/` that holds its output, so every number can be regenerated.

Summary

The paper makes four claims: (1) dips in a common "attention" input plus NMDA gating reproduce the On/Off statistics of PFC decoding (Figs. 1-2); (2) On/Off statistics depend on the decoded population, and the model predicts how (Fig. 2); (3) errors grow with time Off, and the behaviour gives a trace of ~120 ms, matching NMDA and not facilitation (Fig. 3); (4) a spiking ring network reproduces intermittent coding, and population activity keeps the cue's shape during Off states (Fig. 4).

The modelling is careful, the code is clean and the published numbers reproduce exactly from `results/`. But the synthetic ground-truth tests show that **the analyses behind the two most important claims (3 and 4e) cannot tell the proposed mechanism from its alternatives**:

- **Fig. 3b-d.** An STP model in which Off states *never* cause errors still gives the paper's results: total Off time and Off length significant, number of Off states not, and a "best tau_N" of 80-180 ms. That model only needs a few percent of trials in which the memory fails for any other reason (encoding failure, a wrong item, or a random loss), because a lost memory is read out as Off. The fitted tau tracks the fraction of such trials (182 ms at 1%, 118 ms at 4%, 78 ms at 16%), not any time constant. Conversely, with the paper's own fitted NMDA model (tau = 150 ms) as ground truth, the effect is 4x smaller than in the data, and the paper's pooled fit returns tau = 1,030 ms.
- **Fig. 4e.** A ground truth in which the memory is completely *absent* from spiking during silences gives an Off shape similarity of 0.86-0.89, as high as On (0.89-0.96). The scale-free Procrustes measure turns any residual signal in the Off-labelled points (11-33% of them are not truly silent) into a full-shaped ring.
- **Fig. 2.** The top-vs-random shift is reproduced by every variant of the dynamics tried, including one that loses the memory in 87% of trials, and one variant predicts the six untouched populations slightly better than the fitted model. The shift follows from the read-out (z = c dr + eta with c calibrated per population), not from the dynamics. The calibration target itself depends on Off frequency (mu_on falls from 4.1 to 1.2 as the dip rate rises at a fixed gain).
- **Real data.** Error trials are already decoded worse at the cue-response peak (100-400 ms, z 2.31 vs 2.58, p = 0.001, 25 sessions), before any silence could outlast an NMDA trace. Part of the error-Off association is therefore not caused by Off states.
- **Fig. 3a.** The "memory lost from 400 ms" cut-off is set by the scoring rule. After a 500 ms silence, a modest response-period input still restarts the *correct* pool in 100% of trials (90% at 600 ms, >50% at 1 s).

Claims (1) and the qualitative part of (2) hold. Claims (3) and the Fig. 4e part of (4) need either different analyses or much weaker wording. See "What would make the claims hold" at the end.

Figure by figure

Fig. 1: dips produce Off states; NMDA carries the memory

Claim. Brief dips silence both pools; NMDA gating biases which pool restarts; On/Off statistics depend strongly on the read-out gain.

Assessment. The mechanism is clear and the example is correct in the code. Points:

1. **The memory score is a hidden-variable threshold, not a behavioural readout.** `ratemodel.memory_correct` scores "held" when s_c - s_u > 0.05 over the last 100 ms. Methods say a strong response-period input would restart the pool with more gating "(not shown)". It should be shown, because it changes Fig. 3a (below).
2. **98.4% retention is a fit constraint, not a result.** `fit.objective` adds 2e4 * max(0, 0.98 - mem)^2. The abstract and Discussion present near-perfect retention as an outcome.
3. **Restart depends on the feed-forward overshoot** (w_ff, tau_ff = 15 ms). This hand-chosen ingredient is essential to the mechanism, yet appears only in Methods. Show what happens without it.
4. **The dip-free first second** of Fig. 1a is illustrative only (stated in the caption). Fine, but the Discussion's "while attention is sustained, the memory is held without any Off state (Fig. 1a)" uses that illustration as evidence.

Fig. 2: On/Off statistics depend on the decoded population; the model predicts it

Reproduced (`review/check_claims.py` → `out/check_claims.txt`): every top-vs-random comparison gives p = 6e-8. That is the smallest p a two-sided exact Wilcoxon test can reach with 25 sessions, so the test is at its floor. All-unit statistics match the text (Off 3.25/trial, median 112 ms; On 2.10/trial, median 134 ms).

Issues.

1. **The empirical claim is expected under any account** in which the decoder is noisy. A weaker sample is at chance more often. It supports "measured Off fraction is an upper bound", but it does not support persistent activity over activity-silent coding. In an activity-silent model, a random-20 decoder would also show more Off states than a top-20 decoder.
2. **The "prediction" is mostly built into the read-out.** `review/synth_readout_generic.py` → `out/synth_readout_generic.txt`:
   - At a fixed gain, the decoding strength mu_on used to calibrate c falls from 4.08 to 1.17 as the dip rate goes from 2 to 20/s, while frac_off goes from 0.06 to 0.58. In the data, mu_on and frac_off are correlated at rho = -0.90 across sessions. Calibrating c to each population's mu_on therefore already carries information about its Off frequency.
   - With the same per-population calibration, five models with different dynamics all reproduce the shift (top Off < random Off, top On > random On, all three k). The variants are: fitted, dip rate /2, x1.5, dips twice as long, depth 0.4. The summed loss over the six predicted populations is 62.4 for the fitted model, 61.4 for dip rate x1.5 and 66.2 for depth 0.4. Even dips twice as long (memory held in 13% of trials) gives the shift. The prediction does not single out the fitted dynamics.
3. **Quantitative misses** that the text calls a match. For top-k, the fitted model gives 2.5-2.7 Off states per trial against 3.2-3.5 in the data, and frac_off 0.25-0.28 against 0.33-0.36. For random-5, On frac is 0.03 against 0.07. The caption should report the per-population loss (Supp. Fig. 3g already has it) and say where the model misses.
4. **Read-out noise shares rho are typed in** (`readout.RHO`, BUGS A4), and there are three different gains for the same population (BUGS A1). Both are small, but compute them in one place.
5. *Checked and fine.* The model's On threshold is the 95th percentile of all noise-cluster masses, while the data use a per-trial maximum statistic. On the same noise these give 19-24 vs 22-23. Switching the model to the max statistic changes n_on from 1.86 to 1.92 and the loss from 9.2 to 8.5 (`review/check_on_threshold.py`).

Fig. 3: errors increase with time Off; the behaviour points to tau ~ 120 ms

This is the paper's most original claim, and the one the tests undermine most.

Reproduced: tau = 123 ms, bootstrap 101-190 ms, p0 = 0.887 (`out/check_claims.txt`).

3a, single-dip test. `review/synth_single_dip_probe.py` → `out/synth_single_dip_probe.txt`. With the paper's score, the NMDA model holds the memory through 300 ms silences and loses it from 400 ms on. But at 400 ms and beyond, neither pool is active at the end of the delay: the network is silent, not overwritten. Probed by a response-period input (common input 1.15 for 150 ms), the correct pool wins in 100% of trials after 400-500 ms silences, 90% after 600 ms, 68% after 700 ms and still 51% vs 37% wrong after 1 s (1.3 gives 98%, 92%, 74%, 56%). So the "sharp 400 ms limit (about 2.5 tau_N)" depends on the 0.05 threshold, and it disagrees with the paper's own statement that a response input recovers the memory. The STP model needs no probe: it restarts spontaneously after every dip up to 1 s.

3b, GLMM. Two problems:

1. **Reverse causality.** In *every* model, a memory that is lost for any reason is read out as Off for the rest of the delay. So "errors come with more Off time" is predicted by NMDA, STP, encoding failure, wrong-item memory and disengagement alike. The paper's claim that facilitation "predicts no such relation" is true only for STP models with no other source of error.
2. **"Length, not number" does not single out a decaying trace.** One loss produces one long Off state, so it raises Off time and mean Off length but not the number of Off states. The four statistics are strongly correlated (Spearman: frac_off vs off_len 0.82, vs frac_on -0.77) and are tested in separate models.

Ground truth (`review/synth_behaviour.py` → `out/synth_behaviour.txt`). The synthetic data have the data's session structure (25 sessions, trial counts, monkeys, error rates), and states come from the paper's own read-out (all-unit gain and rho):

world	memory loss caused by Off?	beta total Off (p)	beta Off length (p)	beta number of Off (p)	tau, paper's pooled fit [boot 95%]	tau, session p0
data	?	0.31 (1e-5)	0.22 (1e-4)	0.01 (0.8)	123 [101-182]	158
NMDA, tau 80	yes	0.59 (2e-14)	0.35 (6e-14)	0.14 (2e-7)	81 [78-84]	82
NMDA, tau 150 (paper's fitted model)	yes	0.08 (0.03)	0.04 (0.1)	0.02 (0.4)	**1030** [151-1124]	158
NMDA, tau 300	yes	-0.03 (0.3)	-0.05 (0.07)	0.05 (0.04)	427 [201-819]	403
STP, lapses only	no	-0.03 (0.3)	-0.02 (0.4)	0.03 (0.3)	403 (flat)	688
STP, 8% encoding failures	**no**	0.93 (2e-11)	0.42 (2e-12)	0.28 (2e-8)	85 [81-89]	95
STP, 8% wrong item held	**no**	0.87 (4e-11)	0.49 (2e-13)	0.08 (3e-3)	87 [85-91]	97
STP, 8% lost at a random time	**no**	0.54 (1e-10)	0.39 (4e-11)	0.02 (**0.4**)	97 [92-101]	99

- **The STP random-loss world reproduces Fig. 3b exactly**: total Off and Off length significant, number of Off states not, total On negative. Yet in that world Off states never cause errors.
- **Fig. 3c-d: the tau fit measures the fraction of failed trials, not a trace.** `review/synth_tau_vs_failures.py` → `out/synth_tau_vs_failures.txt` runs the STP random-loss world with failure fraction F (two seeds each): F = 1%: 167-182 ms; 2%: 140-143; **4%: 118 (the paper's 123)**; 8%: 98-99; 16%: 78-79 ms. A world without any trace returns the paper's value as soon as about 4% of trials fail for any reason.
- **The paper's own model does not produce the data's effect.** With the fitted NMDA model (tau 150 ms, 98.5% held) as ground truth, beta(total Off) is 0.08, against 0.31 in the data. Fitting tau to the data therefore asks for a model whose On/Off statistics are no longer those of the fit.

- **The pooled estimator is not reliable.** At the true tau of 150 ms the pooled fit returns 1,030 ms. One p0 per session recovers 158 ms. Both recover 80 ms; at 300 ms the estimate is weakly identified.

Real-data checks (`review/tau_fit_within_session.py` → `out/tau_fit_within_session.txt`):

1. **Pooling across sessions.** With one p0 per session (only within-session variation informs tau), the data give **158 ms** (profile 147-169), not 123. This is the same GLMM logic as Fig. 3b, which z-scores within sessions.
2. **Monkeys disagree.** H: 80 ms (77-85); J: 185 ms (172-199); A: 151 ms (136-1,061). H's and J's intervals do not overlap. Error rates are 28% (H), 9% (A) and 1.8% (J), so the pooled estimate is dominated by monkey H. Under the failure-fraction reading above, this ordering is what one expects.
3. **The tau sweep is not monotonic** (`results/ratemodel/tau_sweep.csv`). P(held) is about 1 in every Off bin for tau = 200-750 ms, but the model breaks at tau >= 1 s: P(held) is 0.67 *in every bin* at 1,500 ms. The curve family therefore contains no working long-trace model. "Far shorter than the time scale of facilitation" is never tested by this fit; the upper bound of 190 ms only says that curves for 200-750 ms are flat.
4. **Small effect.** P(correct) falls from 0.88 to 0.79 across Off bins, and trials without any Off state still have 11% errors (vs 15% with Off). Most errors do not depend on Off time; p0 = 0.887 absorbs them.

Data test of reverse causality. Error trials are already decoded worse 100-400 ms after the cue (p = 0.001), before any Off state could have erased a 120-150 ms trace. See "Real-data error time course" below.

Code notes. Error trials are decoded with decoders fitted on all correct trials and z-scored with the correct-trial mean and SD, while correct trials use leave-one-out. Both are out of sample, but any global rate difference in error trials (engagement) shifts their z. Report the pre-cue z of error vs correct trials as a control.

Fig. 4: spiking ring network

1. **"Minimal changes" understates them.** J+ goes from 1.62 to 3.0, a random fraction of input goes through a broad footprint, the cue is 500 pA, g_NMDA is lowered, and a hand-made slow overshoot is added because a fast one erased the memory. The dips are hand-set (7/s, 150 ms, 50%) and differ from the rate-model fit. The network is not fitted, so Fig. 4b is qualitative.

2. **17% of network trials lose the memory** and are excluded ("only trials in which the network held the memory (83%)"). The data's 15% errors are not all memory losses, so the network's retention is worse than the rate model's 98% and is not discussed.

3. **The read-out differs from the data pipeline** in three ways: counts instead of z-scored rates, a pooled shuffle null for z, and a quadratic decoder. The rate model has a validation of its read-out (Supp. Fig. 1); the network has none.

4. **Fig. 4c** is fine as a replication of Panichello's analysis, which it also inherits. The preferred cue is chosen on On + Off data from the same cells, and the shuffle subtraction corrects only the alignment bias.

5. **Fig. 4e cannot show that the memory stays in activity during Off states.** `review/synth_population.py` → `out/synth_population.txt` uses 330 Poisson neurons on a ring, 8 cues x 60 trials, and silences drawn with the fitted dip statistics. The paper's spiking read-out labels states on half the units and `population.analyse` runs on the other half:

ground truth	tuning amplitude	mu_on	Off precision	sim On	sim Off	sim Off (core only)	sim Pre
memory **absent** during silences	8 Hz	1.8	0.67	0.89	**0.89**	0.83	0.15
memory **absent** during silences	14 Hz	4.0	0.89	0.96	**0.86**	0.71	0.31
memory reduced to 30%	8 Hz	2.5	0.68	0.94	0.93	0.88	0.25
memory reduced to 30%	14 Hz	5.6	0.88	0.98	0.95	0.72	0.20

- When the memory is truly absent during silences, Off shape similarity stays near On. It cannot tell "absent" from "reduced". Two reasons: 11-33% of Off-labelled points fall outside true silences (label noise from half-population decoders and 100 ms windows), and Procrustes ignores scale, so a small residual signal averaged over many trials gives a full-shaped ring.
- Restricting Off to "core" points (>= 50 ms from the run edges) lowers the similarity only to 0.71-0.83.
- *Pre-cue windows* in the recordings are centred up to -10 ms, so they extend to +40 ms after cue onset (the network stops at -50 ms). In the synthetic data, with zero-latency neurons, this alone gives pre-cue similarity of 0.15-0.31. With PFC latencies the leak is probably small, but use the same edge for both.
- The sentence "even when the decoder labelled the memory Off, population activity still held a detectable code" therefore does not follow. Off-labelled moments contain mislabelled On activity; that holds whatever the memory mechanism.

Supplementary figures

- **S1 (read-out validation).** It is honest, but it validates the shortcut against a decoder fed by the model itself. The undercount of about one On state per trial is large: the data have 2.1 On states per trial, so this is about 50%. It should be propagated into how Fig. 2's On predictions are read.
- **S2 (no-dip control).** Convincing that dips are needed for the top-unit decoders.
- **S3 (STP model).** Depression was switched off "to give the STP model the best chance". The 112-vs-172 ms justification cannot be reproduced, because that code was removed (BUGS B2). The STP memory is scored on facilitation u, but in the probe test the STP model restarts spontaneously, so this does not change results.

Text and consistency problems in the manuscript

- Leftover suggestion-mode text: "SixSeven parameters", "fixed atbounded between 50 and 150 ms ... the fit placed it at the upper bound", "far feweralmost no On states", "fewer than the datafew", " (; summed fit loss", "assumed in of the fitted model". The Fitting paragraph still describes tau_N as fitted, while the code fixes it (`ratemodel.FIXED`, BUGS C3).
- The title in the Doc ("...through NMDA-sustained persistent activity") differs from the README and repo title.
- The abstract's "trace of ~120 ms" vs the fixed 150 ms in both models vs 158 ms with session p0: say which estimate, and give the per-monkey spread.
- Methods: pre-cue SD 1.06 cannot be reproduced from `results/` (BUGS B1).
- Methods, behaviour: "13,962 trials with at least one Off state" applies to *all four* GLMMs (`glmm_single` filters `n_off > 0` for every statistic). Say so; the frac_off model drops the trials with zero Off.

What would make the claims hold

1. **Fig. 3, test causality, not association.**

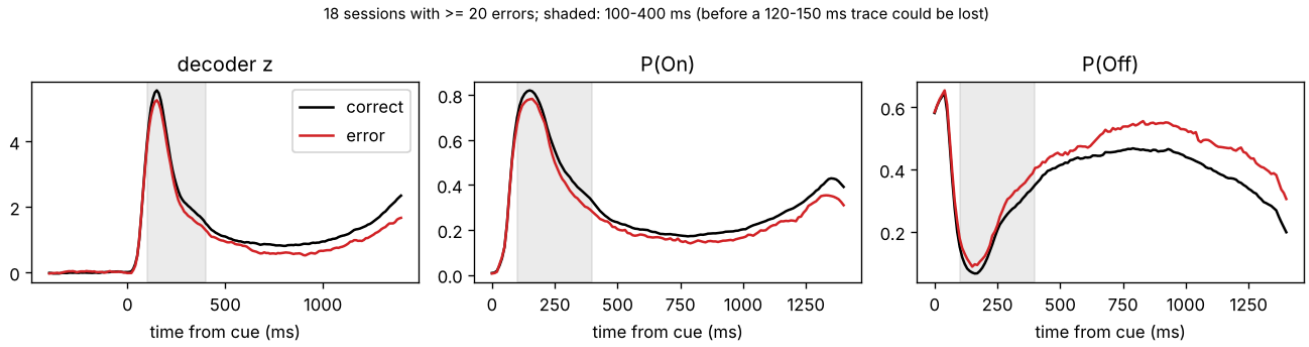
- Use the time course: show that error trials are decoded normally right after the cue and diverge only after a long Off state (aligned to the first Off state longer than ~300 ms), with recovery afterwards in correct trials but not in error trials.
- Show that decoding strength right after the cue does not differ between error and correct trials, or include it as a covariate.
- Fit tau with session-level p0 and report per-monkey estimates.
- Fit the same pipeline to an STP model *with* a realistic error source (`synth_behaviour.py` does this) and show the data are better explained by the NMDA curve shape, not just by a slope.
- Extend the sweep with a long-trace model that still works at tau >= 1 s (refit, or scale beta).

2. **Fig. 4e.** Run the same shape analysis on a ground truth with truly silent Off states (as here), and on the STP rate/spiking model, to set the null. Then compare the *size* of the Off centroids, not only their shape, or estimate the Off-label precision.
3. **Fig. 2.** Present the shift as a consequence of read-out noise, and the model's quantitative fit to each population as the test. Show alternative dynamics fail on that fit (the variants here do not).
4. **Fig. 3a.** Score memory with a response-period probe, or justify the 0.05 threshold. Report the curve under both scores.

Real-data error time course

`review/data_error_timecourse.py` → `out/data_error_timecourse.{txt,csv,npz}`, plot `out/data_error_timecourse.png` (`plot_error_timecourse.py`). The all-units decoder is run exactly as in `scripts/recordings.py`, for all 25 sessions.

window	correct z	error z	Wilcoxon over sessions
pre-cue (-400 to -10 ms)	-0.004	0.039	p = 0.21 (no offset: the error-trial z-scoring is not biased)
early, 100-400 ms	2.58	2.31	**p = 0.0013** (median error/correct ratio 0.885)
delay, 500-1,400 ms	1.01	0.64	p = 5e-5 (ratio 0.624)



- Error trials are already decoded worse at the **peak of the cue response (~150 ms)**, and their P(Off) is already higher by 250-400 ms. That is before any silence could have outlasted a 120-150 ms trace (the single-dip test needs about 400 ms). Part of the error-Off association therefore comes from what happened at encoding or from engagement, and not from Off states erasing a trace.
- The divergence then grows through the delay (ratio 0.885 → 0.62). This is also consistent with a delay-period loss, so the data are a *mixture*.
- So the Fig. 3 claim needs this decomposition. Add early decoding strength (100-400 ms z) as a trial-level covariate in the GLMM and in the tau fit, or restrict the analysis to error trials with normal encoding. The synthetic worlds above show how much a few percent of encoding failures shift the tau estimate.

Files

script	tests	output
`check_claims.py`	headline numbers of Figs. 2-3 from `results/`	`out/check_claims.txt`
`check_on_threshold.py`	On threshold: model vs data definition	`out/check_on_threshold.txt`
`tau_fit_within_session.py`	tau fit pooled vs per session vs per monkey (real data)	`out/tau_fit_within_session.txt`
`synth_behaviour.py`	ground-truth worlds through the GLMM and the tau fit	`out/synth_behaviour.{txt,csv}`, `out/synth_trials_*.csv.gz`
`synth_tau_vs_failures.py`	tau fit vs fraction of failed trials, no trace	`out/synth_tau_vs_failures.{txt,csv}`
`synth_single_dip_probe.py`	Fig. 3a with a response-period probe	`out/synth_single_dip_probe.{txt,csv}`
`synth_readout_generic.py`	Fig. 2 calibration leakage and specificity	`out/synth_readout_generic.txt`, `out/synth_readout_*.csv`
`synth_population.py`	Fig. 4e on a truly silent ground truth	`out/synth_population.{txt,csv}`
`data_error_timecourse.py`, `plot_error_timecourse.py`	when error trials diverge (real data)	`out/data_error_timecourse.{txt,csv,npz,png}`
`common.py`	shared helpers	
`callgraph.py`	static call graphs: all functions, per module, per figure (Appendices A-B of the PDF)	`out/callgraph/*.{svg,png,dot}`, `out/callgraph/calls.json`
`build_pdf.py`	this review as a PDF with the call-graph appendices	`REVIEW.pdf`

Run from the repository root after `pip install -e .`, e.g. `python review/synth_behaviour.py`. Times on 4 cores: the checks take seconds; the synthetic scripts take 5-20 min each; the data time course takes about 1 h (it needs `../panichello`).

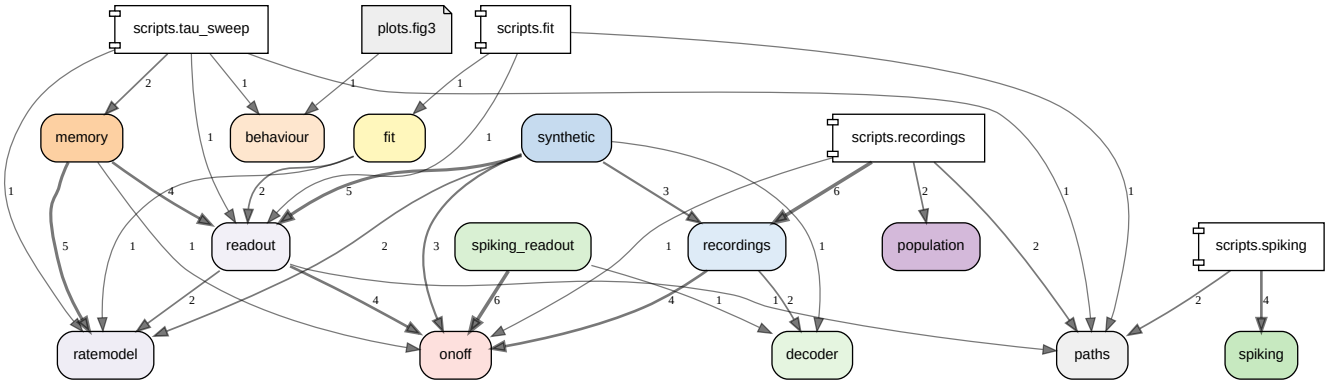
Appendix A. Call graphs of the code

Built by static analysis of the source (`review/callgraph.py`, Python `ast`; nothing is run): 103 functions and methods of `nmdaint/`, `scripts/` and the `draw()` entry points of `plots/`, with 133 call edges. A call is resolved through the package's import aliases (`from . import onoff`, `from nmdaint import readout`), local names, `self.` methods, instances built in the same function (`S = recordings.Session(...)`), a method name that is unique in its module (`net.cue_current`), class instantiation (`Class()`), and functions passed by reference (`joblib.delayed(f)`). Nested functions and lambdas count as their enclosing function, and the `if __name__ == '__main__'` block of a script is shown as `__main__`. Calls to `numpy`, `scipy`, etc. are left out. Limitations: calls through variables holding functions (other than `delayed`), `getattr` and dictionary dispatch are not seen; the plotting internals are omitted.

Colours: one per `nmdaint` module; white component boxes are `scripts/`; folded-corner boxes are `plots/`. An arrow A → B means A calls B.

A.1 Module dependencies

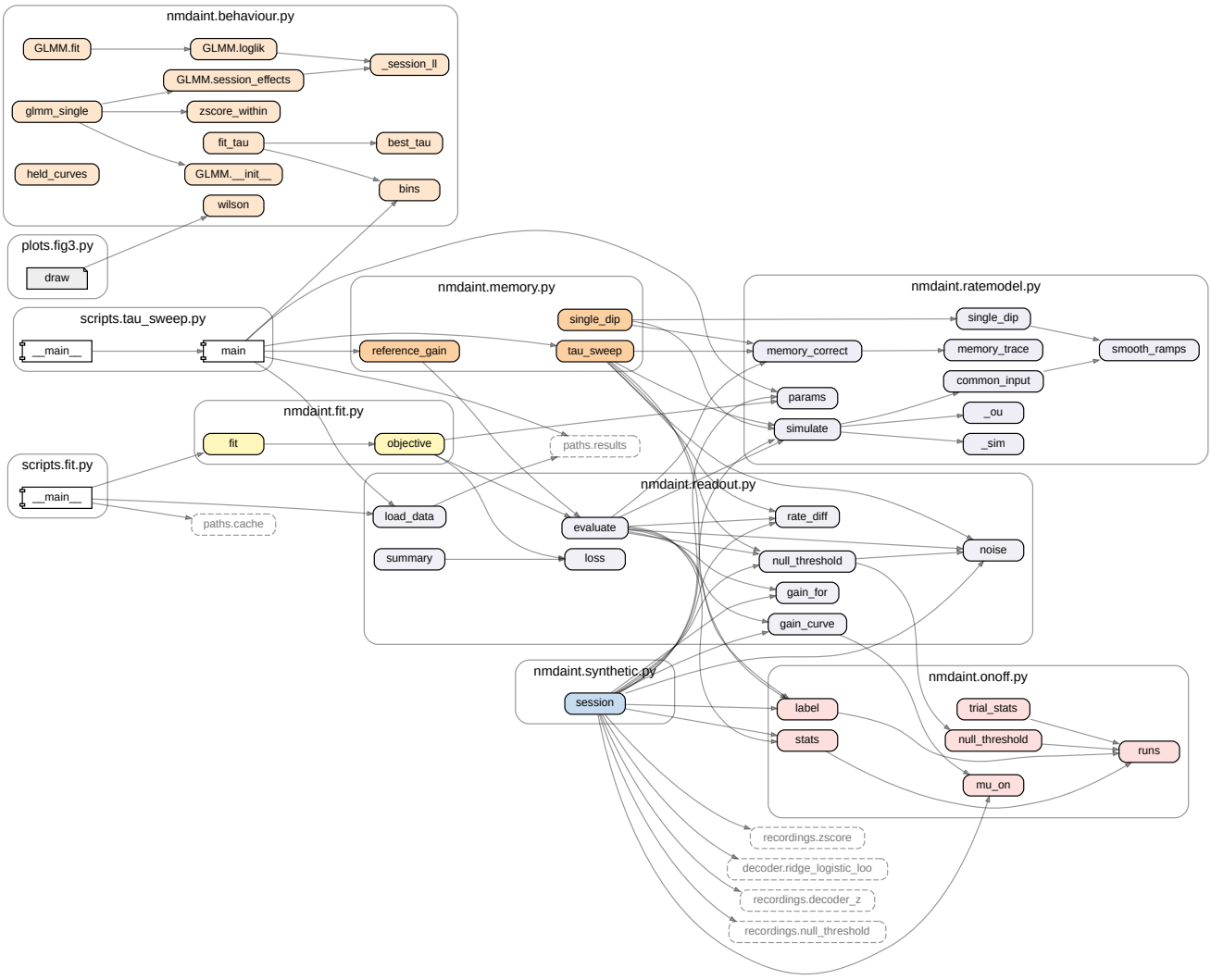
Edge labels give the number of distinct (caller, callee) function pairs between two modules.



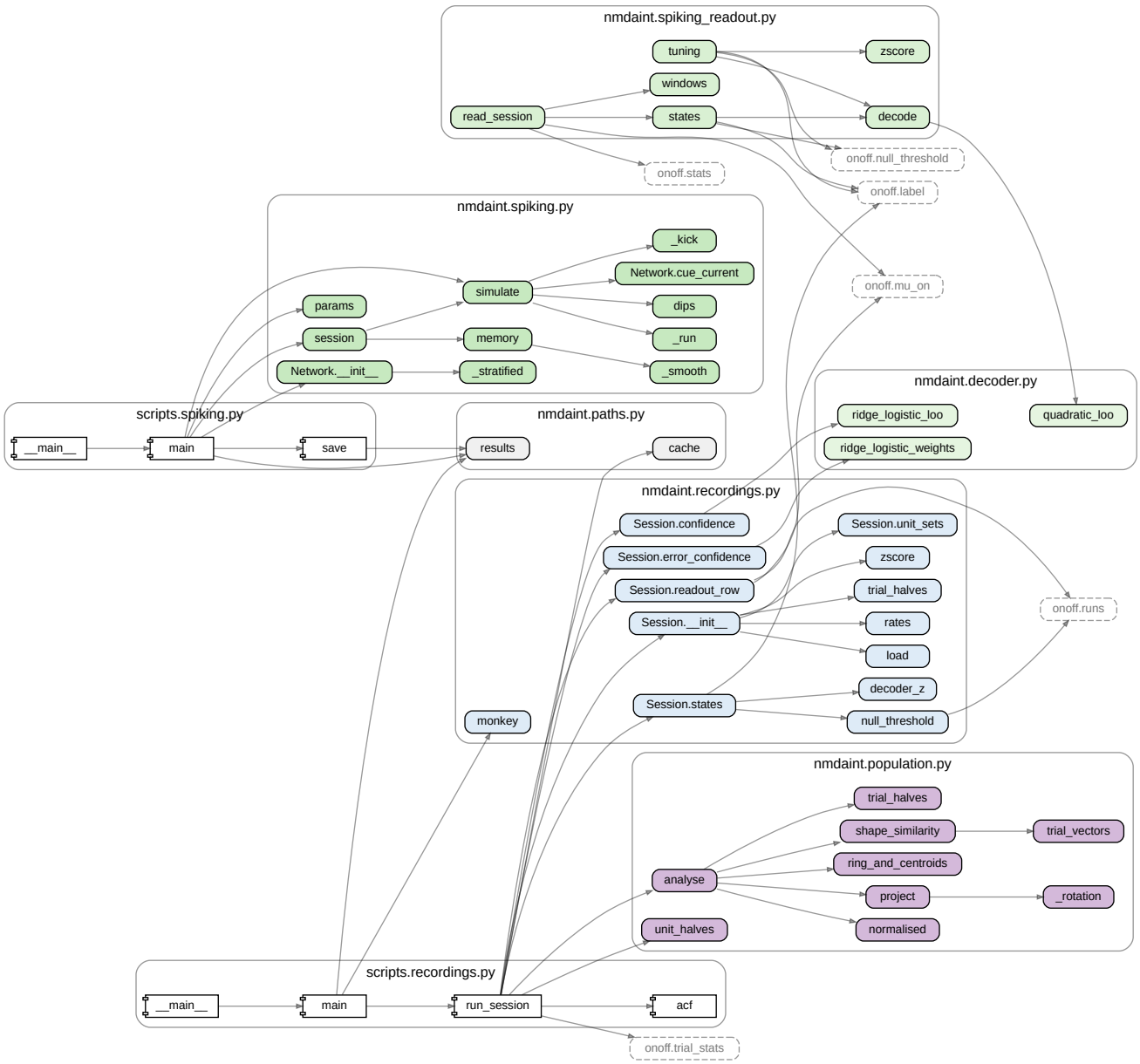
Three shared cores: `onoff` (state labelling and statistics, used by the recordings, both models' read-outs and the spiking read-out), `readout` (the rate-model read-out, used by the fit, the memory tests, the synthetic validation and the tau sweep) and `ratemodel`. The recordings pipeline (`recordings`, `decoder`, `population`) is reached only from `scripts/recordings.py` and from `synthetic`, whose validation runs the recordings' decoder on model-driven spikes.

A.2 Functions: rate model, read-out, fit, behaviour (and the scripts that use them)

Dashed grey boxes: functions of other modules (A.3). `plots/*.draw` take data and call no package function, except `plots.fig3.draw` (Wilson intervals); `behaviour.held_curves` is called only by the Fig. 3 notebook.



A.3 Functions: recordings, decoders, population geometry, spiking network (and their scripts)

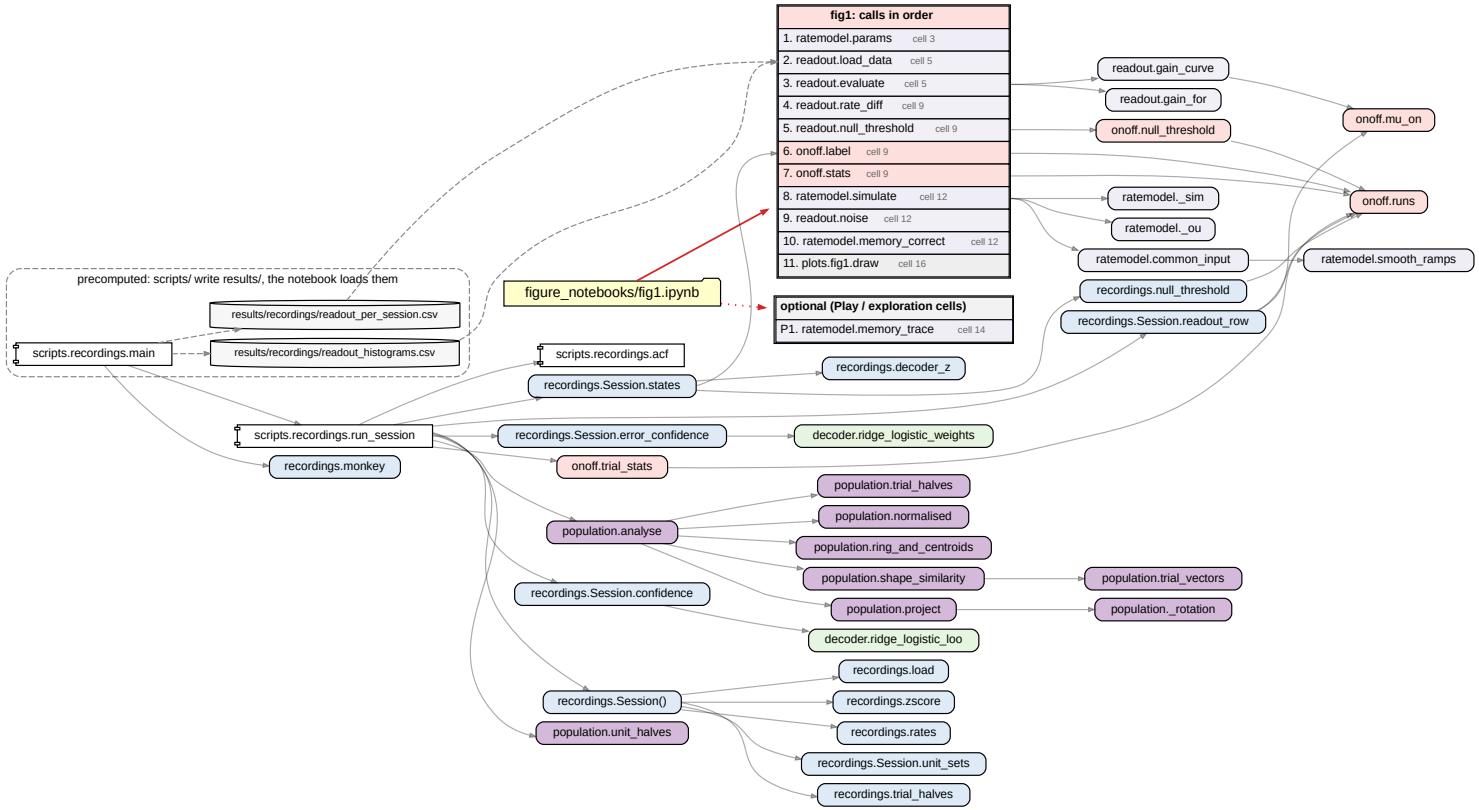


The single graph of all functions is `review/out/callgraph/callgraph_all.svg`; the edges as JSON: `review/out/callgraph/calls.json`.

Appendix B. What each figure runs, in order

For each figure notebook (`figure_notebooks/`): the package functions the notebook calls, numbered in execution order (first call of each function; within a statement, arguments before the call), each expanded into everything it calls. Cells under a "Play" or "optional" heading are exploration, not part of the figure; they are listed separately in grey. Dashed: the `results/` files the figure loads (directly or inside a library function such as `readout.load_data`) and the `scripts/` functions that wrote them, with what those scripts call. These are the heavy, precomputed steps behind the figure.

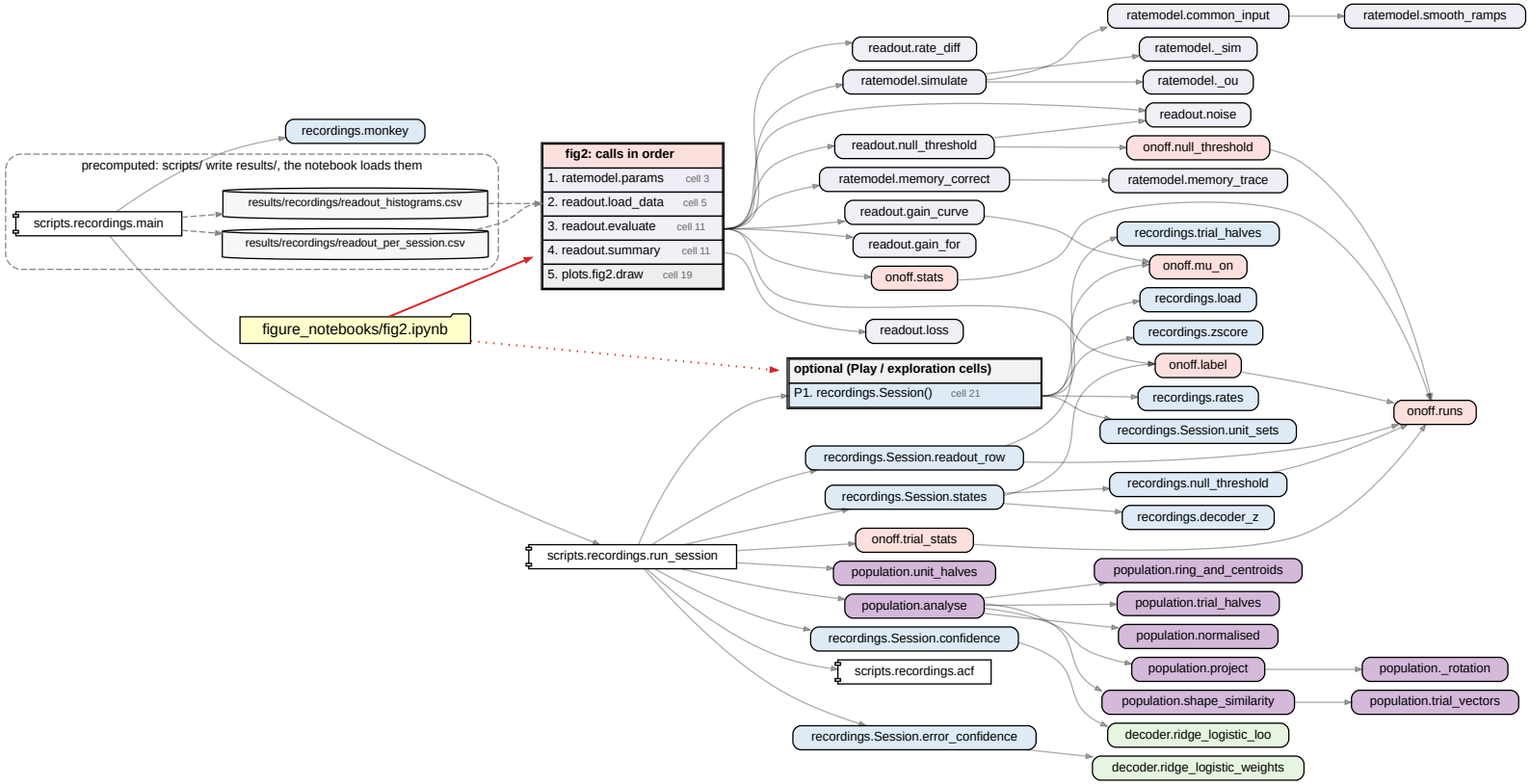
Figure 1 (`figure_notebooks/fig1.ipynb`)



#	function called by the notebook	cell
1	`ratemodel.params`	3
2	`readout.load_data`	5
3	`readout.evaluate`	5
4	`readout.rate_diff`	9
5	`readout.null_threshold`	9
6	`onoff.label`	9
7	`onoff.stats`	9
8	`ratemodel.simulate`	12
9	`readout.noise`	12
10	`ratemodel.memory_correct`	12
11	`plots.fig1.draw`	16
P1	`ratemodel.memory_trace`	14

loaded file	written by	used by
`results/recordings/readout_per_session.csv`	`scripts.recordings.main`	figure
`results/recordings/readout_histograms.csv`	`scripts.recordings.main`	figure

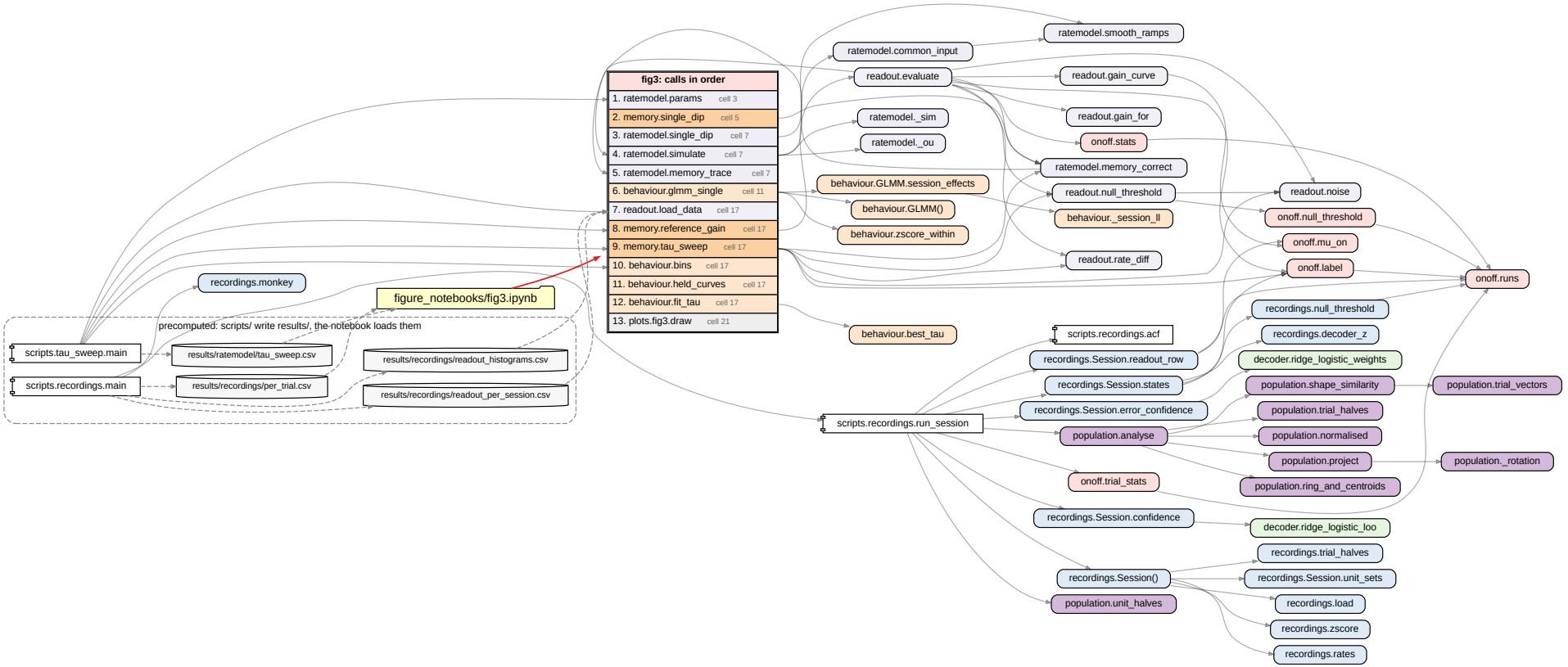
Figure 2 (`figure_notebooks/fig2.ipynb`)



#	function called by the notebook	cell
1	`ratemodel.params`	3
2	`readout.load_data`	5
3	`readout.evaluate`	11
4	`readout.summary`	11
5	`plots.fig2.draw`	19
P1	`recordings.Session()`	21

loaded file	written by	used by
`results/recordings/readout_per_session.csv`	`scripts.recordings.main`	figure
`results/recordings/readout_histograms.csv`	`scripts.recordings.main`	figure

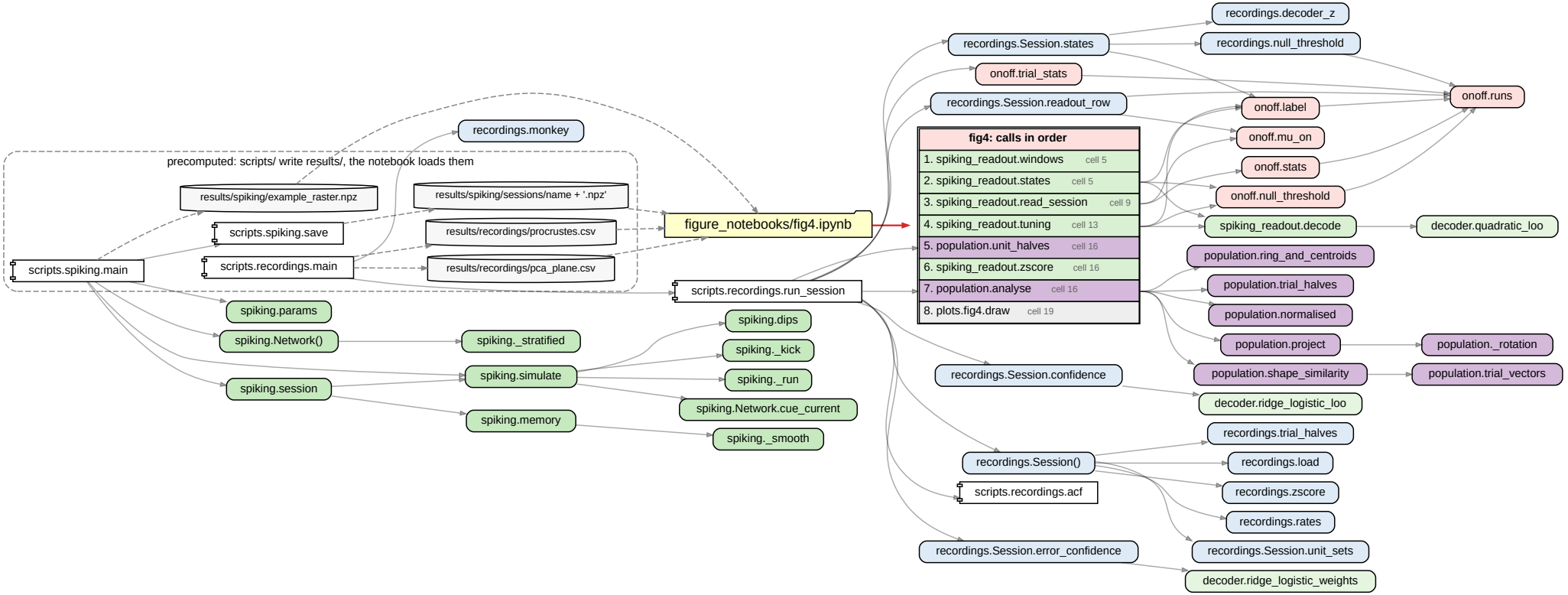
Figure 3 (`figure_notebooks/fig3.ipynb`)



#	function called by the notebook	cell
1	`ratemodel.params`	3
2	`memory.single_dip`	5
3	`ratemodel.single_dip`	7
4	`ratemodel.simulate`	7
5	`ratemodel.memory_trace`	7
6	`behaviour.glmm_single`	11
7	`readout.load_data`	17
8	`memory.reference_gain`	17
9	`memory.tau_sweep`	17
10	`behaviour.bins`	17
11	`behaviour.held_curves`	17
12	`behaviour.fit_tau`	17
13	`plots.fig3.draw`	21

loaded file	written by	used by
`results/recordings/per_trial.csv`	`scripts.recordings.main`	figure
`results/ratemodel/tau_sweep.csv`	`scripts.tau_sweep.main`	figure
`results/recordings/readout_per_session.csv`	`scripts.recordings.main`	figure
`results/recordings/readout_histograms.csv`	`scripts.recordings.main`	figure

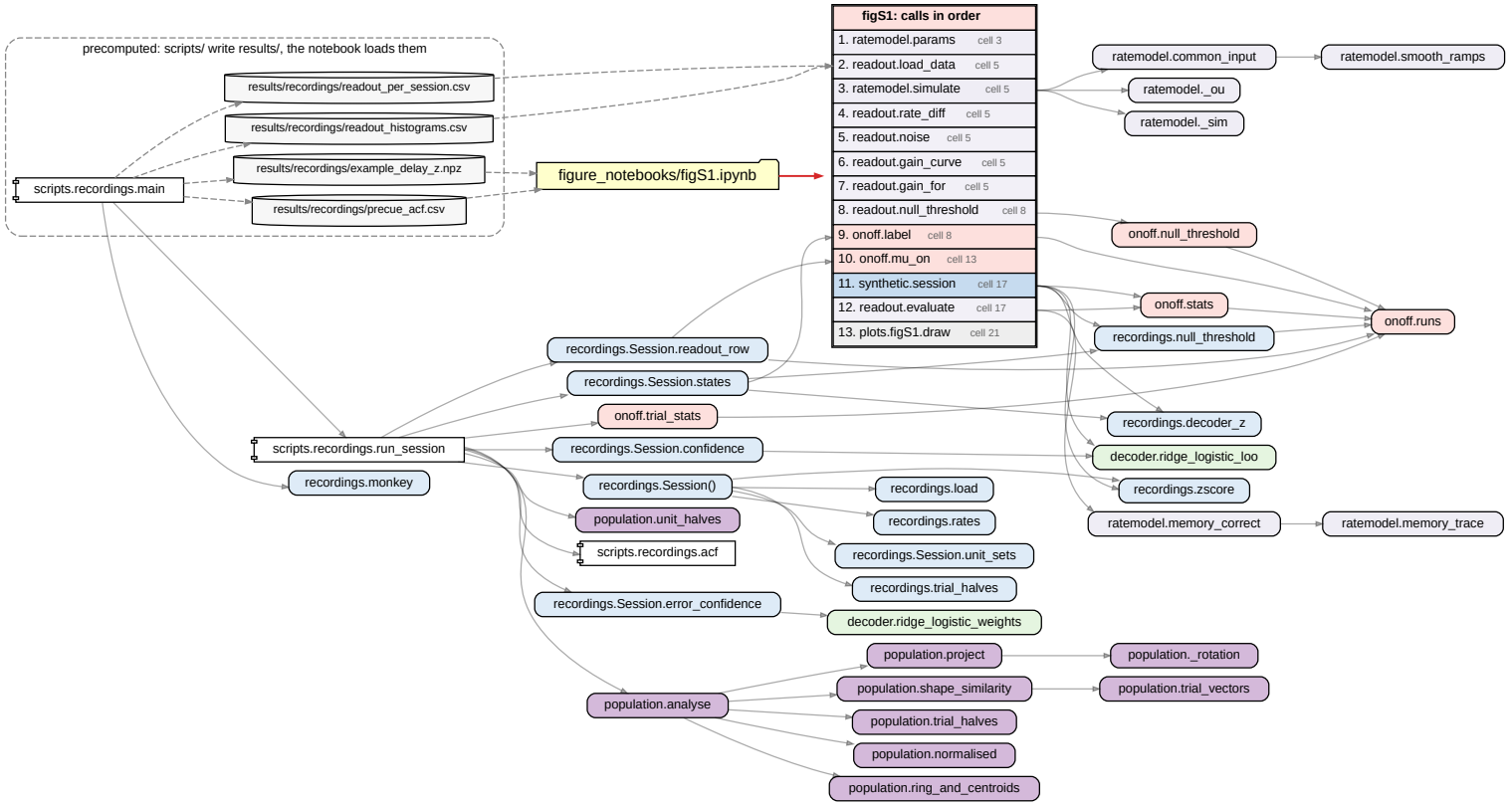
Figure 4 (`figure_notebooks/fig4.ipynb`)



#	function called by the notebook	cell
1	`spiking_readout.windows`	5
2	`spiking_readout.states`	5
3	`spiking_readout.read_session`	9
4	`spiking_readout.tuning`	13
5	`population.unit_halves`	16
6	`spiking_readout.zscore`	16
7	`population.analyse`	16
8	`plots.fig4.draw`	19

loaded file	written by	used by
`results/spiking/sessions/name + '.npz'`	`scripts.spiking.save`	figure
`results/spiking/example_raster.npz`	`scripts.spiking.main`	figure
`results/recordings/procrustes.csv`	`scripts.recordings.main`	figure
`results/recordings/pca_plane.csv`	`scripts.recordings.main`	figure

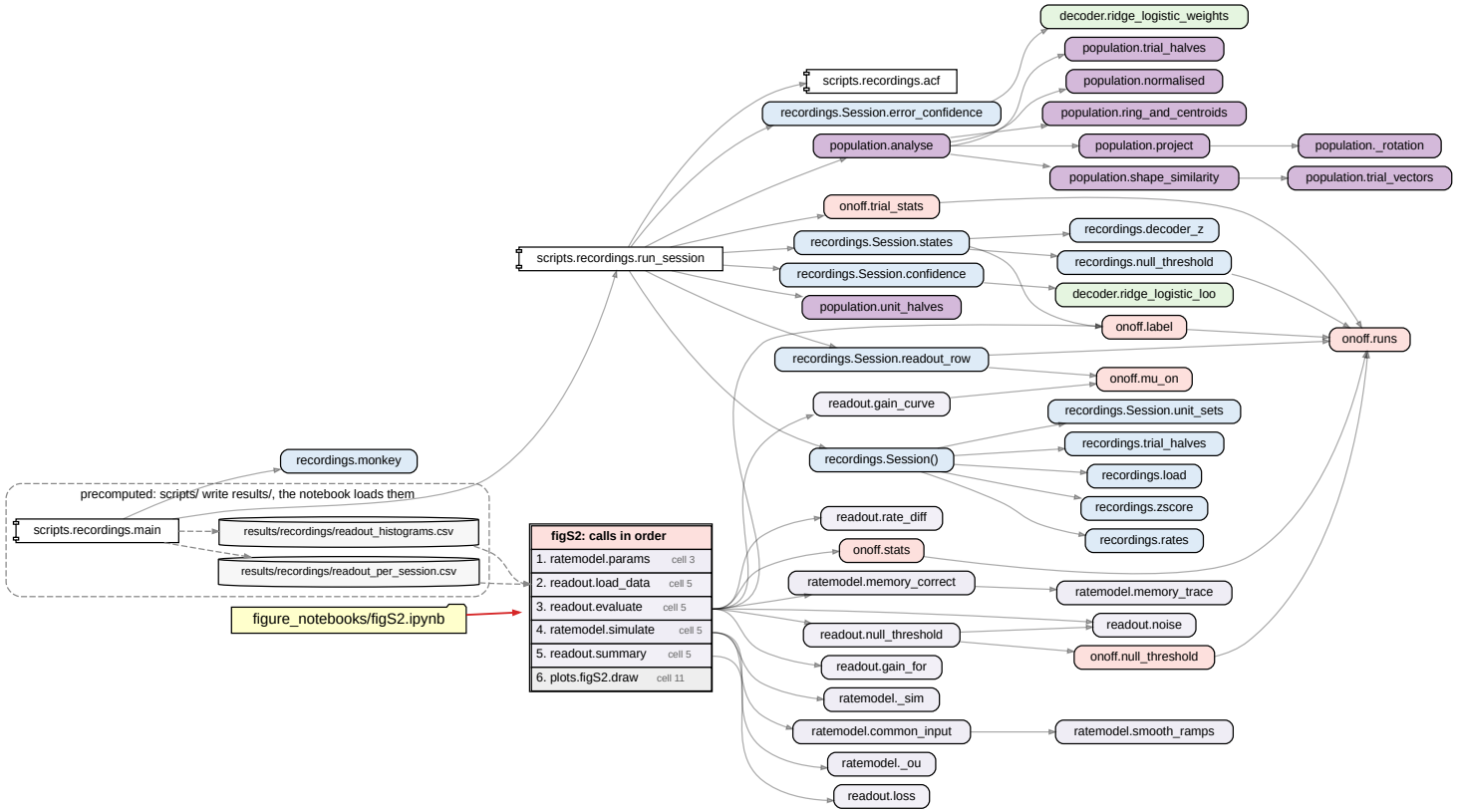
Supplementary Figure 1 (`figure_notebooks/figS1.ipynb`)



#	function called by the notebook	cell
1	`ratemodel.params`	3
2	`readout.load_data`	5
3	`ratemodel.simulate`	5
4	`readout.rate_diff`	5
5	`readout.noise`	5
6	`readout.gain_curve`	5
7	`readout.gain_for`	5
8	`readout.null_threshold`	8
9	`onoff.label`	8
10	`onoff.mu_on`	13
11	`synthetic.session`	17
12	`readout.evaluate`	17
13	`plots.figS1.draw`	21

loaded file	written by	used by
`results/recordings/precue_acf.csv`	`scripts.recordings.main`	figure
`results/recordings/example_delay_z.npz`	`scripts.recordings.main`	optional
`results/recordings/readout_per_session.csv`	`scripts.recordings.main`	figure
`results/recordings/readout_histograms.csv`	`scripts.recordings.main`	figure

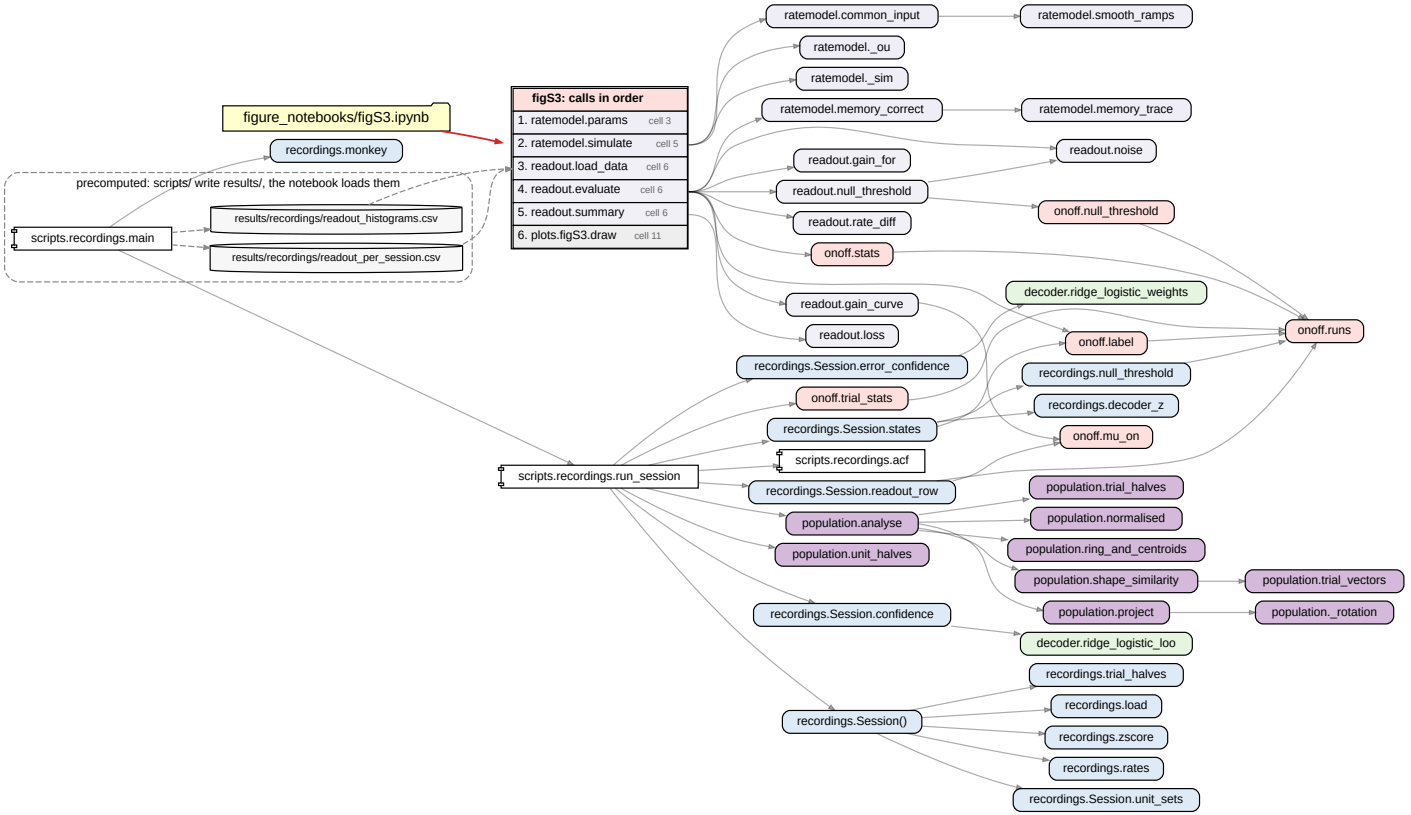
Supplementary Figure 2 (`figure_notebooks/figS2.ipynb`)



#	function called by the notebook	cell
1	`ratemodel.params`	3
2	`readout.load_data`	5
3	`readout.evaluate`	5
4	`ratemodel.simulate`	5
5	`readout.summary`	5
6	`plots.figS2.draw`	11

loaded file	written by	used by
`results/recordings/readout_per_session.csv`	`scripts.recordings.main`	figure
`results/recordings/readout_histograms.csv`	`scripts.recordings.main`	figure

Supplementary Figure 3 (`figure_notebooks/figS3.ipynb`)



#	function called by the notebook	cell
1	`ratemodel.params`	3
2	`ratemodel.simulate`	5
3	`readout.load_data`	6
4	`readout.evaluate`	6
5	`readout.summary`	6
6	`plots.figS3.draw`	11

loaded file	written by	used by
`results/recordings/readout_per_session.csv`	`scripts.recordings.main`	figure
`results/recordings/readout_histograms.csv`	`scripts.recordings.main`	figure